2024 4th International Conference on Computer, Communication, Control & Information Technology (C3IT) | 979-8-3503-8912-8/24/$31.00 ©2024 IEEE | DOI: 10.1109/C3IT60531.2024.10829414

Performance Evaluation of Application Layer IoT Protocols: MQTT and CoAP

Manika Debnath
Department of Computer Science & Engineering
Supreme Knowledge Foundation Group of Institutions
Mankundu, West Bengal, India
debnathmonika62@gmail.com
ORCID: 0009-0002-7301-7850

Subrata Modak
Department of Computer Science & Engineering
Supreme Knowledge Foundation Group of Institutions
Mankundu, West Bengal, India
subratamodak.cs@ gmail.com
ORCID: 0000-0002-5403-8536

Dhrubasish Sarkar
Computer Science & Information Technology
Supreme Institute of Management and Technology
Mankundu, West Bengal, India
dhrubasish@inbox.com
ORCID: 0000-0002-7418-2922

***Abstract*—The Internet of Things (IoT) is one of the most intriguing technological revolutions of the last few years due to its rapid expansion. The advancement of IoT applications is dependent on standard and real-time communication protocols. The IoT protocols are designed to enable reliable messaging. However, the selection of a standard and efficient messaging protocol is a difficult and intimidating task for any organization. One of the main challenging issues facing the IoT is wireless sensor networks (WSNs). The devices that make up these networks have low levels of memory, processing power, and energy. Certain restricted protocols must be in place to function in WSNs due to their constrained nature. In the context of a business application for monitoring, this paper shows the experimental performance of two protocols: Message Queue Telemetry Transport (MQTT) and Constrained Application Protocol (CoAP).**

***Index Terms*—MQTT,CoAP, IoT, CoRE, HTTP, Contiki OS, Cooja simulator**

I. Introduction

The IoT, which combines M2M communication, has recently become the most well-liked Internet advancement.It consists of an enormous number of networked sensing, actuating, processing, and communication devices that exchange data with each other via a range of radio technologies, such as Wi-Fi, Bluetooth, 6LowPAN (IPv6 over Low Power Personal Area Networks), Zigbee (which is based on the IEEE 802.15.4 Standard), and Wi-Fi. In order to address the additional issues posed by settings with a large number of heterogeneous, resource-constrained devices, IoT communication architectures and protocols have been created[1]. Furthermore, Transmission Control Protocol/Internet Protocol (TCP/IP) supports IoT end-to-end services. In Fig. 1., The Protocols Stack for IoT System shows different types of Protocols, and the interoperability of TCP/IP is hindered in the IoT scenarios by the requirements of the widely distributed heterogeneous IoT environments, including the need to support low latency, time-sensitive data, and power constraints [2][3]. Broker-based publish/subscriber communication protocols and architectures, which form the basis of IoT scenarios, distinguish between data sources (producers) and receivers (consumers) through abstraction. About networking semantics and performance, in particular, packet loss and latency—that is, the time it takes for a request to finish—are essential topics of study for this research, which aims to provide an in-depth comprehension of the various IoT protocols for communication and architectures that are currently in use[3][4]. This paper discusses the main characteristics, advantages, and disadvantages of IoT applications by outlining many protocols and protocol frameworks.The work provides a comprehensive comparison of the solutions under study. Two specific solutions are evaluated in terms of performance in this work: MQTT and CoAP. Packet loss and the amount of time it takes for requests to complete (latency) are used to assess performance[5].

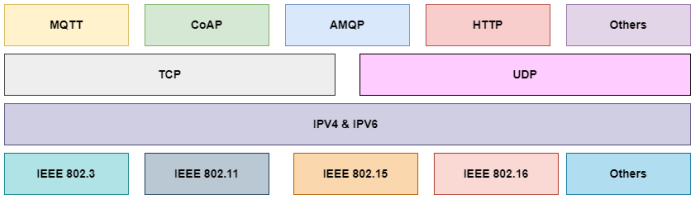

Fig. 1. Protocol Stack for IoT system

II. Related Works

An easy-to-use publish/subscribe communications protocol for M2M and IoT is called MQTT (Message Queue Telemetry Transport) [6]. While MQTT clients are aggregated networked devices, the MQTT server is sometimes called a broker. The client-server protocol MQTT transmits and receives messages using the publish and subscribe concept. Data is ”published” when it is sent by a client (device) to the MQTT broker. Furthermore, in reaction to a client or device’s data request, an MQTT broker ”subscribes” [6]. MQTT is the most widely utilized IoT communications protocol, as IoT resources are limited devices [7]. MQTT is an easy-to-use, open-source protocol that works well for small-scale applications like the IoT, which has minimal power, memory, and bandwidth [7]. In response to a GET request made using the Constrained Application Protocol (CoAP), for example, the server provides a single resource. It may send a response message

with varying payload lengths[2],[5]. In low-power settings, CoAP is superior to MQTT [8]. However, this reduced power results in a longer delay [10]. The authors examined the transmission energy usage of CoAP and MQTT_SN, the two primary application layers of Io protocols, and also investigated message length. These two light application methods function similarly once the total energy usage for transmission is understood. On the other hand, MQTT_SN client nodes are superior to CoAP clients since they are more surface-level. The application determines how these protocols are used since MQTT_SN needs an infrastructure that includes allocating the broker system. [12] MQTT can be considered if you work from remote areas with low network power and bandwidth, limited resources, and unstable networks.The two most popular protocols for M2M connection are MQTT and CoAP. The drawbacks of MQTT are scalability, inadequate security encryption, and slower transmit cycles than CoAP. MQTT is used in many different domains: Enterprise-level applications, social networking, healthcare, home automation, lightweight apps, and utilities [13].

III. Application Layer IoT Protocols Stack:

A. MQTT

MQTT((Message Queuing Telemetry Transport) is among the earliest M2M communication protocols introduced in 1999[9]. It was created by Arlen Nickel of Arcom Control Systems Ltd (Eurotech) and Andy Stanford-Clark of IBM. In Fig. 2. and Fig. 3., it is a publish/subscribe messaging protocol made for M2M communications that are lightweight and possible in networks with limitations. When a MQTT

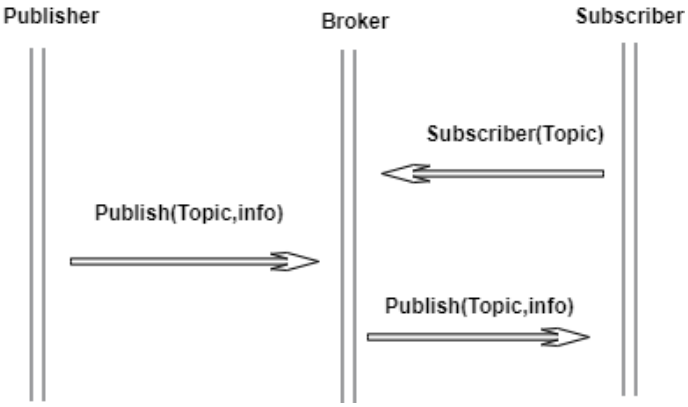

Fig. 2. Messages exchanged by MQTT protocol.

client publishes messages to a MQTT broker, other clients may subscribe to them or they may be saved for a future subscription. By minimizing message header overhead and guaranteeing effective data transfer, MQTT employs a lightweight binary format. It is, therefore, appropriate for IoT devices that have limited power and bandwidth. Because of its efficient messaging model, support for reliability and scalability, and lightweight design, MQTT is a popular option for IoT applications.

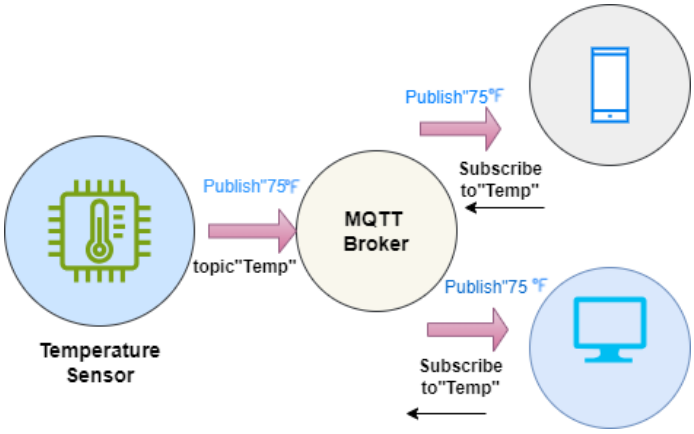

Fig. 3. MQTT Publish-Subscribe model

B. CoAP

Constrained Application Protocol (CoAP) is an additional lightweight communication protocol made especially for IoT devices with constrained processing, memory, and battery life[2]. In Fig. 4., IoT devices can communicate with resources using well-known protocols like GET, POST, PUT, and DELETE.CoAP's RESTful architecture is similar to HTTP. This facilitates the effortless integration of CoAP with current web technologies and APIs[11]. Unlike HTTP, which typically runs over TCP, CoAP operates over UDP (User Datagram Protocol), reducing the overhead associated with connection establishment and maintaining state. UDP is well-suited for IoT applications where low latency and reduced overhead are critical. CoAP works well in IoT applications where effective communication, low power consumption, and resource limitations are critical factors. It is a compelling option for a variety of IoT deployments due to its RESTful architecture, UDP-based communication, and support for resource observation.

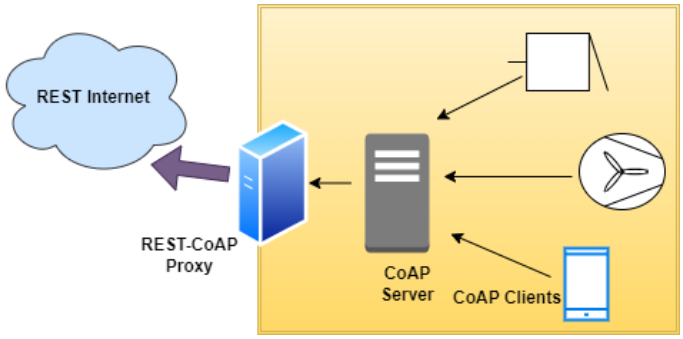

Fig. 4. In a CoAP setting, a client/server protocol

C. COOJA

A simple Java-based simulator called COOJA is designed to simulate sensor networks powered by the Contiki operating system[9]. The main purpose of Cooja is to model and assess wireless sensor network performance. Before implementing

them on actual hardware, it enables researchers and developers to test their apps, protocols, and algorithms in a safe setting. Cooja primarily supports Contiki-based IoT devices [9]. Contiki is well-known for supporting several low-power wireless communication technologies, including 6LoWPAN, IEEE 802.15.4, and Bluetooth Low Energy (BLE). Thus, Cooja is suitable for simulating networks of devices operating on these standards. Cooja offers a simple GUI for building, configuring, and visualizing virtual wireless sensor networks. Network topology can be specified, node parameters can be set, and real-time network behaviour can be observed. Nodes can interact with one another through wireless communication protocols in the realistic simulation environment that Cooja offers. Users can assess the effectiveness of their protocols in a variety of scenarios by simulating factors like packet loss, signal attenuation, and radio interference. Since Cooja is part of the Contiki ecosystem, it seamlessly integrates with Contiki OS development tools and libraries. This integration allows developers to easily port their applications from simulation to real hardware and vice versa. The features of each tool are

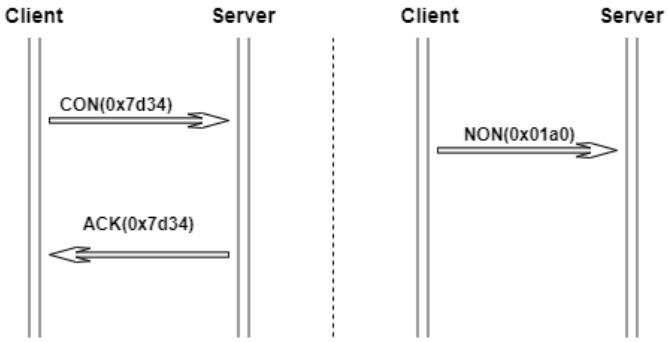

Fig. 5. Messages types exchanged in CoAP

briefly explained here.

• Network - Demonstrates where each network node is located. able to display the current state of all nodes, including those with LEDs, addressable addresses, multiple outputs, mote IDs, etc. We need to add our sensors to this window since it is currently empty.

• Simulation Control - The simulation can be started, stopped, reloaded, or its steps carried out using this panel. It displays the execution time and simulation speed. It indicates that we can complete the tasks many times quicker than they would be completed in real-time.

• Notes -This is a basic notebook for making notes on the simulated environment.

• Mote output - Shows each and every output from the serial port of the nodes. Each simulation node has the option to enable one Mote output window.

• Timeline - A timeline simulation that shows events and messages such as log outputs, channel changes, and LED changes.

Other tools that can be enabled in the Tools menu to appear in addition to the default ones include Breakpoints, Radio Messages, Script Editor, Buffer View, and Motor Duty Cycle.

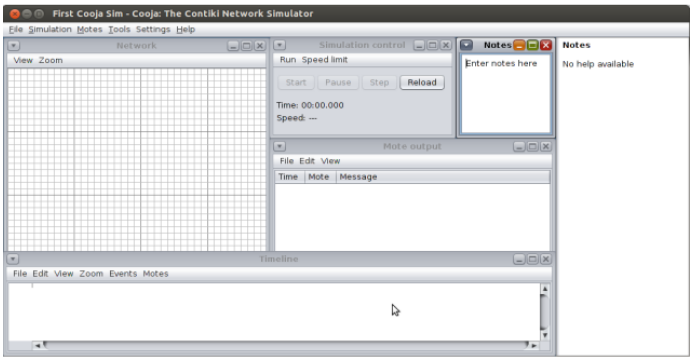

Fig. 6. Interface of cooja simulator

IV. METHODOLOGY

This research compares the efficiency of MQTT with that of CoAP [2] to assess its usefulness for WSNs. Both the reaction time and the server motor's energy usage are considered in the comparison. To assess whether MQTT and CoAP function well, identical tests were run on both protocols. The Cooja simulator and the Contiki operating system are used for the simulations. Contiki is an operating system intended for computers with constrained memory, emphasizing low-power wireless IoT devices. It serves as a basis for networked systems, focusing on efficiency and optimizing environments with limited resources. Contiki's minimum-power REST machine, Erbium (Er), is the foundation for its CoAP implementation. Three different component types made up the test bed: a border router, a CoAP server for creating server-side programs, and a CoAP client that uses the POST method to send non-confirmable messages to the actuators, toggling resources every ten seconds without requiring an ACK and saving a significant amount of energy.

A. MQTT

The development of automation systems for a house also referred to as a "smart house" or "smart accommodation," is known as home automation[11] in Fig. 7. It includes home appliances like ovens, freezers, and washers/dryers as well as the automation and control of lighting, heating, ventilation, air conditioning, and safety. We used temperature and light sensors in our work to observe MQTT's functionality. To implement MQTT with a quality of service level (QoS) of 0, the border router, publisher, and subscriber (client) were utilized as the CoAP clients. The subscribers are set up to transmit a MQTT PINGREQ message every 10 seconds. The broker was the lightweight and inexpensive Small Message Broker (RSMB). It enables communicating across networks with limited processing power, bandwidth, and dependability to and from tiny devices, such as actuators and sensors. Except for building a bridge that enables connections with conventional MQTT brokers like Mosquitto, the RSMB broker resembles conventional MQTT brokers quite a bit.

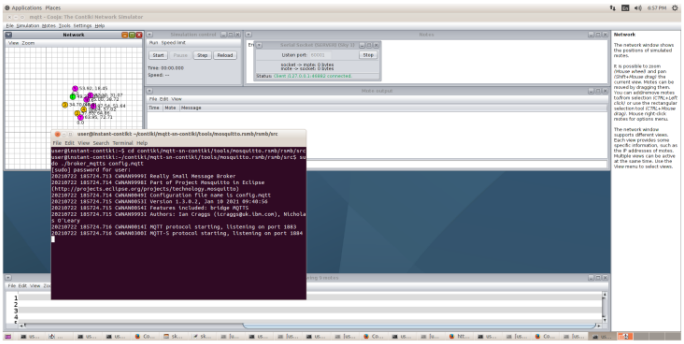

Fig. 7. Home automation scenario by using MQTT

After running the simulation, it showed current readings of Light and Temperature.

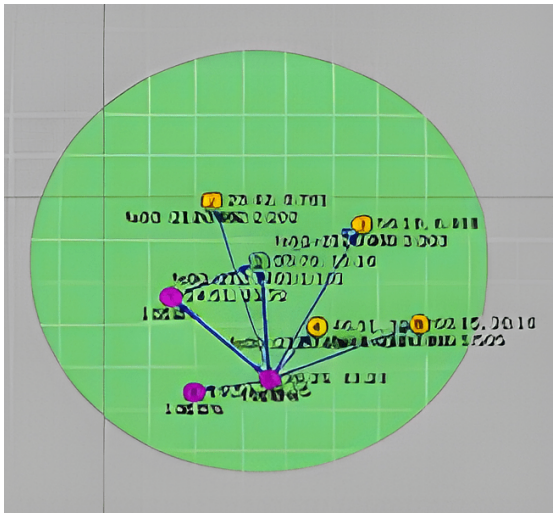

Fig. 8. Messages passed to client nodes from publisher nodes

B. CoAP

The low-power REST machine Erbium (Er) provides the framework for Contiki's CoAP implementation. A border router, a CoAP server for server-side development of applications, and a CoAP client that asks the actuators and toggles resources every ten seconds were the three distinct component types we deployed in the testbed. The CoAP client sends non-confirmable messages via the POST method, eliminating the need for an ACK and saving a significant amount of energy.

By using the above applications, we have analyzed the performance of MQTT and CoAP.

V. Result and Analysis

The simulation time is the amount of time spent on simulating a model. In our work in Fig. 8., Fig. 9. and Fig.10., we have taken a simulation time of 307398 ms. We have examined and compared the power consumption, throughput, and average inter-packet latency for the MQTT and CoAP protocols.

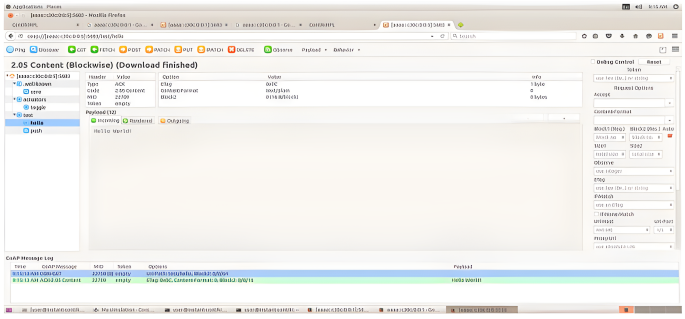

Fig. 9. " Hello world " message passed from server

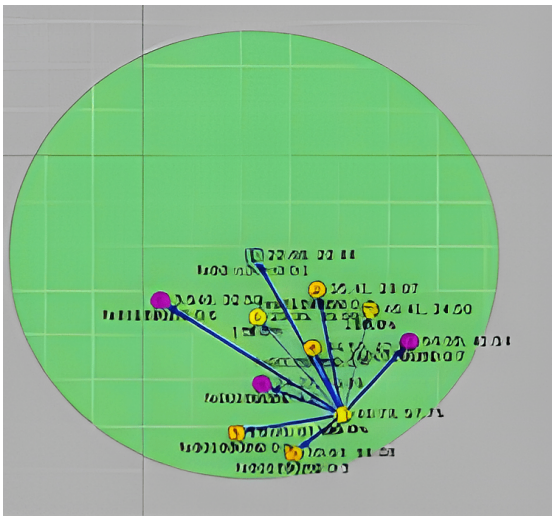

Fig. 10. Message passed to client nodes from server

A. Power consumption

In TABLE I and Fig. 11., the total amount of electrical energy used by every device in the network infrastructure over a given time frame is referred to as power consumption in computer networks. It includes how much energy different parts like switches, routers, computers, servers, and other network accessories use. Managing power consumption in a network is important for optimizing energy efficiency, reducing operational costs, and minimizing environmental impact.

TABLE I
Power consumption value for MQTT and CoAP in (mW)

No of Clients	*MQTT*	*CoAP*
0	0	0
3	1.374	0.713
6	1.575	0.884
9	1.589	1.134
12	1.764	1.173

In TABLE I, we can see that the MQTT protocol consumed more power than CoAP. Therefore, we can say that the power consumption of MQTT is higher than CoAP protocols.

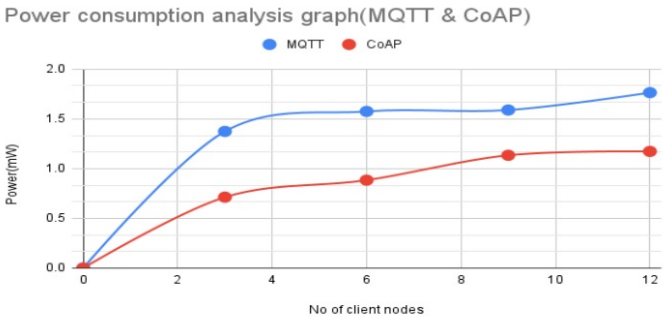

Fig. 11. Graph for power consumption of MQTT and CoAP

B. Average inter- packet time

In a data transmission or network communication, the average interval between successive packets is referred to as the average inter-packet time. TABLE II and Fig. 12., it's a measurement of the typical amount of time that passes between sending one packet and sending the subsequent packet in a series. However, based on average inter-packet time, COAP

TABLE II
AVERAGE INTER-PACKET TIME OF MQTT AND COAP IN SECOND.

No of Clients	*MQTT*	*CoAP*
0	0	0
3	41	40
6	44	41
9	47	33
12	30	34

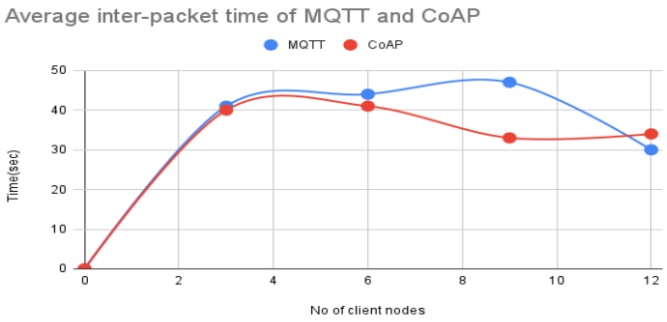

Fig. 12. Average inter-packet time of MQTT and CoAP

produces slightly better result than MQTT.

C. Throughput of MQTT and CoAP

The rate at which a message is successfully delivered across a communication channel in a network is called throughput. It's an important metric to evaluate a network's effectiveness and performance, especially in situations where high data transfer speeds are necessary, like data centres, cloud computing environments, or multimedia streaming services. In TABLE IV and Fig. 13., the throughput of MQTT and CoAP,

TABLE III
THROUGHPUT OF MQTT AND COAP

No of Clients	*MQTT*	*CoAP*
0	0	0
3	0.0374	0.0412
6	0.0374	0.0412
9	0.0187	0.0274
12	0.0187	0.0274

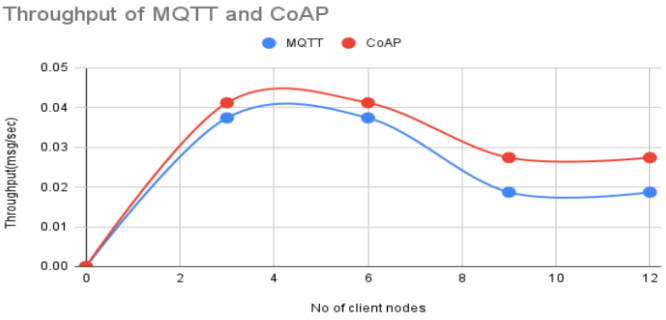

Fig. 13. Graph for throughput of MQTT and CoAP

we can say that the throughput of CoAP is higher than MQTT. So, that means CoAP performed better in this scenario.

D. Average radio duty cycle between MQTT and CoAP

In TABLE IV and Fig. 14., the average radio duty cycle refers to the proportion of time that a radio transmitter or receiver is actively transmitting or receiving data compared to the total time within a given period. This metric is particularly important in wireless communication systems, where devices often need to conserve energy to prolong battery life. On

TABLE IV
AVERAGE RADIO DUTY CYCLE BETWEEN MQTT AND COAP

No of Clients	*MQTT*	*CoAP*
0	0	0
3	0.728	0.325
6	0.882	0.569
9	0.892	0.764
12	0.036	0.798

average radio duty, MQTT outperforms CoAP in terms of results.

VI. CONCLUSION

One of the most significant developments in modern innovation is thought to be the IoT. For this innovation to be developed and improved to be used on a large scale, an IoT system that is secure is essential. In our work, we cover the fundamentals as well as the essential features and specifications of the IoT. The IoT's MQTT and CoAP protocols are examined in this project. We examined these conventions about testing concerns, such as bandwidth, transport, support for quality of service, and so on. Finally, using various metrics, such as power consumption, average inter-packet latency, and

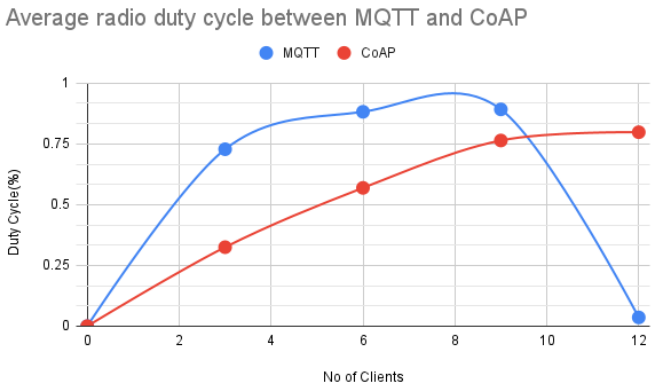

Fig. 14. Graph for Average radio duty cycle between MQTT and CoAP

throughput, we dissected how these conventions were implemented. We observed that each convention performs better on its path and depends on its applications following examination. Based on average power consumption, throughput, and interpacket latency, CoAP outperformed MQTT by a small margin.

REFERENCES

[1] Karagiannis, V.; Chatzimisios, P.; Vazquez-Gallego, F.; Alonso-Zarate, J. A Survey on the Application Layer Protocols for the IoT. Trans. IoT Cloud Comput. 2015, 3, 11–17.

[2] Pohl, Matthias, Janick Kubela, Sascha Bosse, and Klaus Turowski. "Performance evaluation of application layer protocols for the internet-of-things." In 2018 Sixth International Conference on Enterprise Systems (ES), pp. 180-187. IEEE, 2018.

[3] Osterlind, Fredrik, Adam Dunkels, Joakim Eriksson, Niclas Finne, and Thiemo Voigt. "Cross-level sensor network simulation with cooja." In Proceedings. 2006 31st IEEE conference on local computer networks, pp. 641-648. IEEE, 2006.

[4] Hamid, Harth Ghassan, and Zainab T. Alisa. "A survey on IoT application layer protocols." Indonesian Journal of Electrical Engineering and Computer Science 21, no. 3 (2021): 1663-1672.

[5] Bansal, Sharu, and Dilip Kumar. "IoT application layer protocols: performance analysis and significance in a smart city." In 2019 10th International Conference on Computing, Communication and Networking Technologies (ICT), pp. 1-6. IEEE, 2019.

[6] Houimli, Manel, Laid Kahloul, and Siham Benaoune. "Performance analysis of IoT application layer protocol." In Lecture Notes in Real-Time Intelligent Systems, pp. 225-234. Springer International Publishing, 2019.

[7] Kovatsch, M., Duquennoy, S., Dunkels, A., 2011. A low-power coap for contiki, in: 2011 IEEE Eighth International Conference on Mobile Ad-Hoc and Sensor Systems, IEEE. pp. 855–860.

[8] Martí, Mónica, Carlos Garcia-Rubio, and Celeste Campo. 2019. "Performance Evaluation of CoAP and MQTT_SN in an IoT Environment" Proceedings 31, no. 1: 49.

[9] Kodali, R.K., Rajanarayanan, S.C., Boppana, L., Sharma, S., Kumar, A., 2019. Low cost smart home automation system using smart phone, in: 2019 IEEE R10 Humanitarian Technology Conference (R10-HTC)(47129), IEEE. pp. 120–125.

[10] Tandale, Upendra, Bashirahamad Momin, and Deva P. Seetharam. "An empirical study of application layer protocols for IoT." In 2017 International Conference on Energy, Communication, Data Analytics and Soft Computing (ICECDS), pp. 2447-2451. IEEE, 2017.

[11] Singh, U., Ansari, M., 2019. Smart home automation system using the Internet of Things, in 2019 2nd International Conference on Power Energy, Environment and Intelligent Control (PEEIC), IEEE. pp. 144–149.

[12] Silva, Daniel, Liliana I. Carvalho, José Soares, and Rute C. Sofia. "A performance analysis of Internet of Things networking protocols: Evaluating MQTT, CoAP, OPC UA." Applied Sciences 11, no. 11 (2021): 4879.

[13] N. Naik, "Choice of effective messaging protocols for IoT systems: MQTT, CoAP, AMQP and HTTP," in 2017 IEEE International Systems Engineering Symposium (ISSE), Oct. 2017, pp. 1–7, doi: 10.1109/SysEng.2017.8088251.